Lab 7: Fuzzing for Input Validation Bugs

Name: Deeptha S
SRN: PES1UG24CS144
Assigned module: L6_L7_18_atm_withdrawal (ATM Withdrawal)

Objective

To explore fuzz testing as a security an quality assurance technique by identifying and fixing bugs in the provided Python script using the Hypothesis fuzzing library.

Step 1: Install dependencies

Commands: pip install hypothesis and pip install pytest.

```
Collecting sortedcontainers<3.0.0,>=2.1.0 (from hypothesis)
  Using cached sortedcontainers-2.4.0-py2.py3-none-any.whl.metadata (10 kB)
Requirement already satisfied: colorama>=0.4 in C:\Users\shankar\AppData\Roaming\Python\Python314\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=2 in C:\Users\shankar\AppData\Roaming\Python\Python314\site-packages (from pytest) (2.3.1)
Requirement already satisfied: packaging>=24 in C:\Users\shankar\AppData\Roaming\Python\Python314\site-packages (from pytest) (25.0)
Requirement already satisfied: pluggy<2,>=1.6 in C:\Users\shankar\AppData\Roaming\Python\Python314\site-packages (from pytest) (1.7.0)
Requirement already satisfied: pygments>=2.15 in C:\Users\shankar\AppData\Roaming\Python\Python314\site-packages (from pytest) (2.19.2)
Downloading hypothesis-6.170.15-cp314-cp314-win_amd64.whl (691 kB)
   ---------------------------------------- 691.1/691.1 kB 10.1 MB/s  0:00:00
Using cached sortedcontainers-2.4.0-py2.py3-none-any.whl (29 kB)
Installing collected packages: sortedcontainers, hypothesis
   ----------                               1/2 [hypothesis]  WARNING: The script hypothesis.exe is installed in 'C:\Users\shankar\AppData\Roaming\Python\Python314\Scripts' which is not on
 PATH.
  Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.
Successfully installed hypothesis-6.170.15 sortedcontainers-2.4.0

[notice] A new release of pip is available: 26.2.2 -> 26.3
[notice] To update, run: python.exe -m pip install --upgrade pip
```

Figure 1: Installing Hypothesis and pytest

Step 2: Run the buggy application

Command: python atm_withdrawal.py (original, unfixed script). With valid input it works; with invalid input it crashes.

```
shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/buggy_lab7_tmp/Lab 7 (main)
$ python atm_withdrawal.py
==== Running notes_needed ====
Enter an amount (e.g. 2,500): 2500
Number of 100 Notes: 25

==== Running mask_card ====
Enter a card number (e.g. 1234 5678 9012 3456): 1234 5678 9012 3456
Masked Card: XXXX XXXX XXXX 3456

==== Running average_withdrawal ====
Enter withdrawals as CSV (e.g. 500,1000,2000): 500,1000,2000
Average Withdrawal: 1166

shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/buggy_lab7_tmp/Lab 7 (main)
$
```

Figure 2: Original script with valid input (works)

```
shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/buggy_lab7_tmp/Lab 7 (main)
$ python atm_withdrawal.py
==== Running notes_needed ====
Enter an amount (e.g. 2,500): abc
Traceback (most recent call last):
  File "C:\Users\shankar\OneDrive\Desktop\buggy_lab7_tmp\Lab 7\atm_withdrawal.py", line 29, in <module>
    main()
    ~~~~^^
  File "C:\Users\shankar\OneDrive\Desktop\buggy_lab7_tmp\Lab 7\atm_withdrawal.py", line 15, in main
    result = notes_needed(user_input)
  File "C:\Users\shankar\OneDrive\Desktop\buggy_lab7_tmp\Lab 7\atm_withdrawal.py", line 2, in notes_needed
    return int(amount.replace(',', '')) // 100
           ~~~^^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'abc'

shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/buggy_lab7_tmp/Lab 7 (main)
$
```

Figure 3: Original script crashing ith ValueError in notes_needed for amount 'abc'

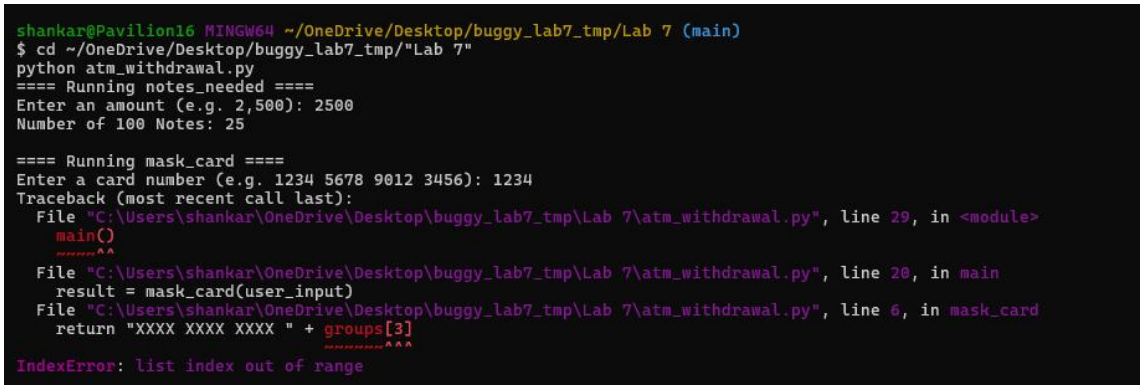


Figure 4: Original script crashing with IndexError in mask_card for card number '1234'

Step 3: Complete test_atm_withdrawal.py using Hypothesis

The three starter tests give st.text() | st.none() to each function and fail if it raises any exception. I added explicit @example inpuuts for known edge cases and property tests: valid input gives the exact expected answer, negative values are rejected, results are always None or a non-negative integer, and a masked card never shows more than its last four digits. The completed file has 11 tests.

Step 4: Run test_atm_withdrawal.py against the buggy script

Command: python -m pytest test_atm_withdrawal.py. Result on the original script: 8 failed, 3 passed.

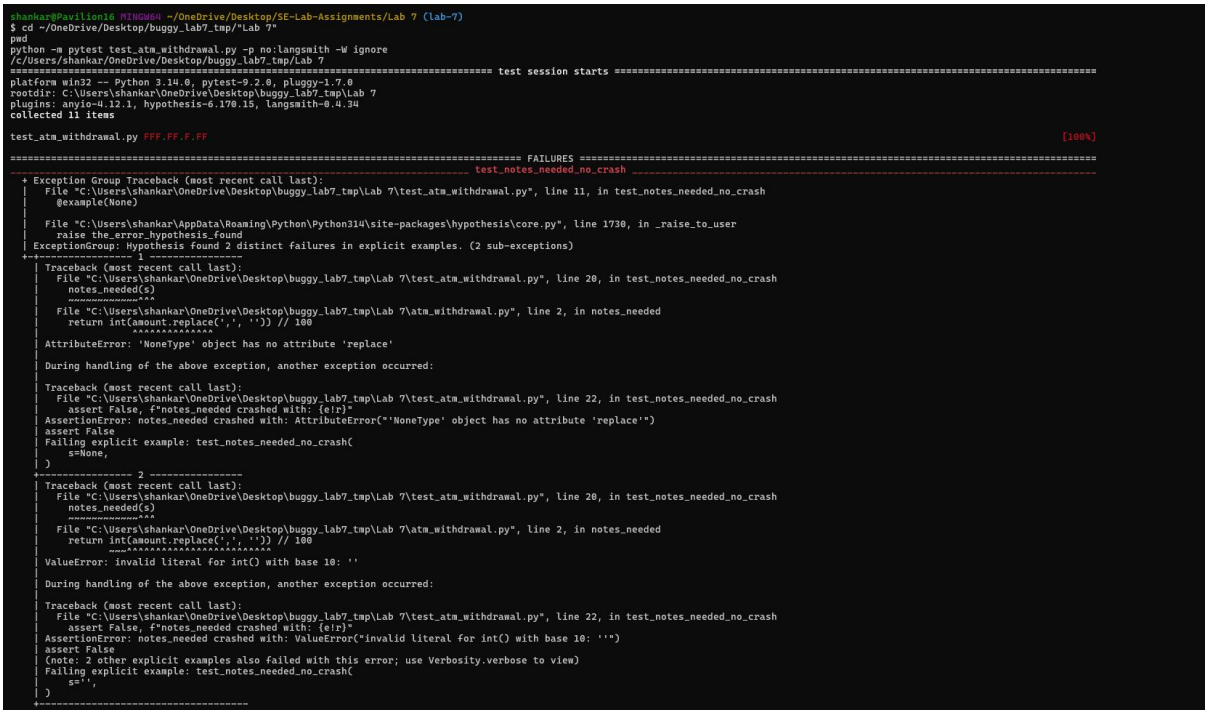


Figure 5: Test cases failingg on the original script (part 1, notes_needed)

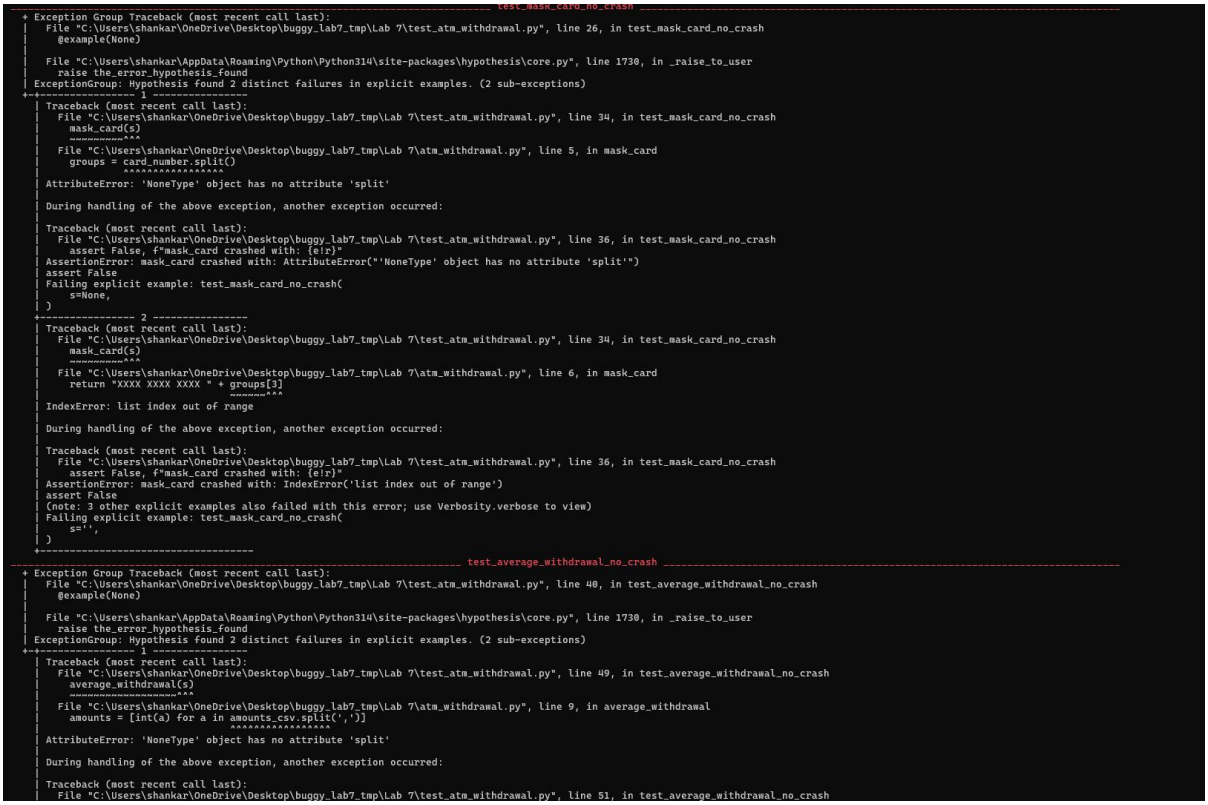


Figure 6: Test cases failing (part 2, mask_card and average_withdrawal)

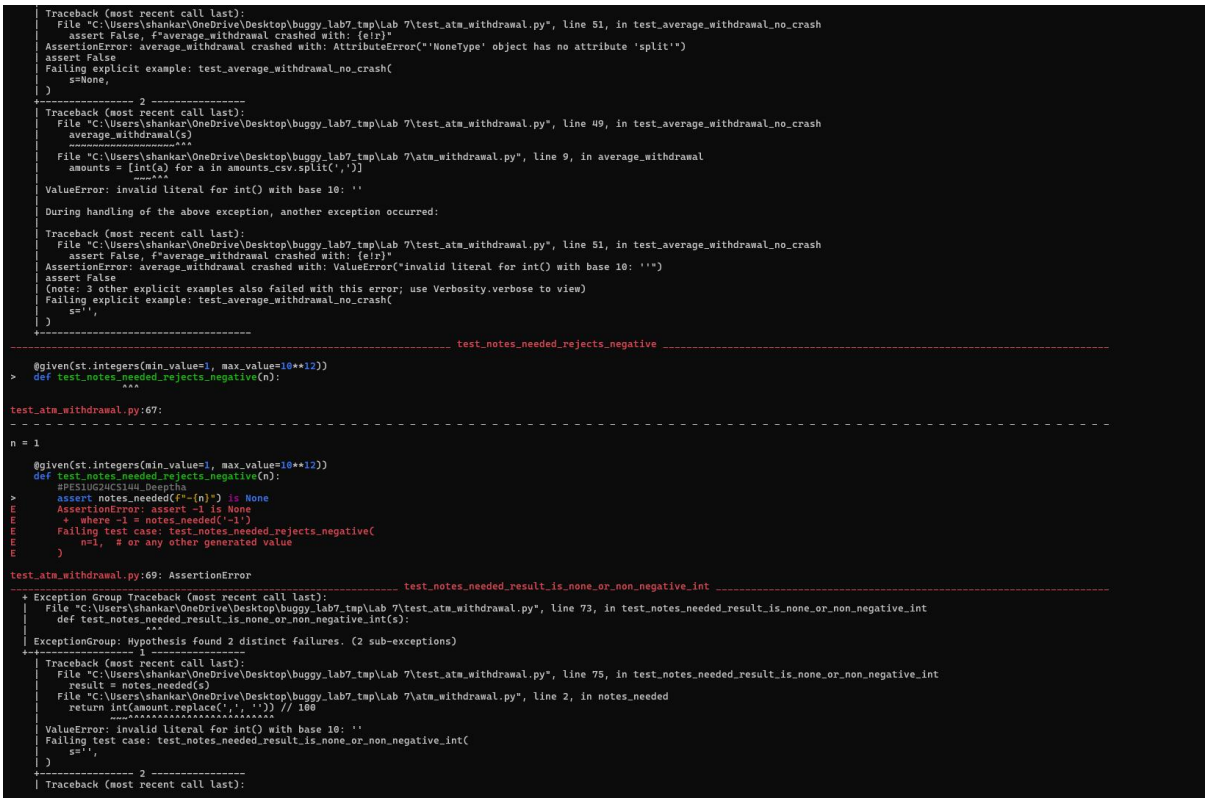


Figure 7: Test cases failing (part 3, negative amount accepted by notes_needed)

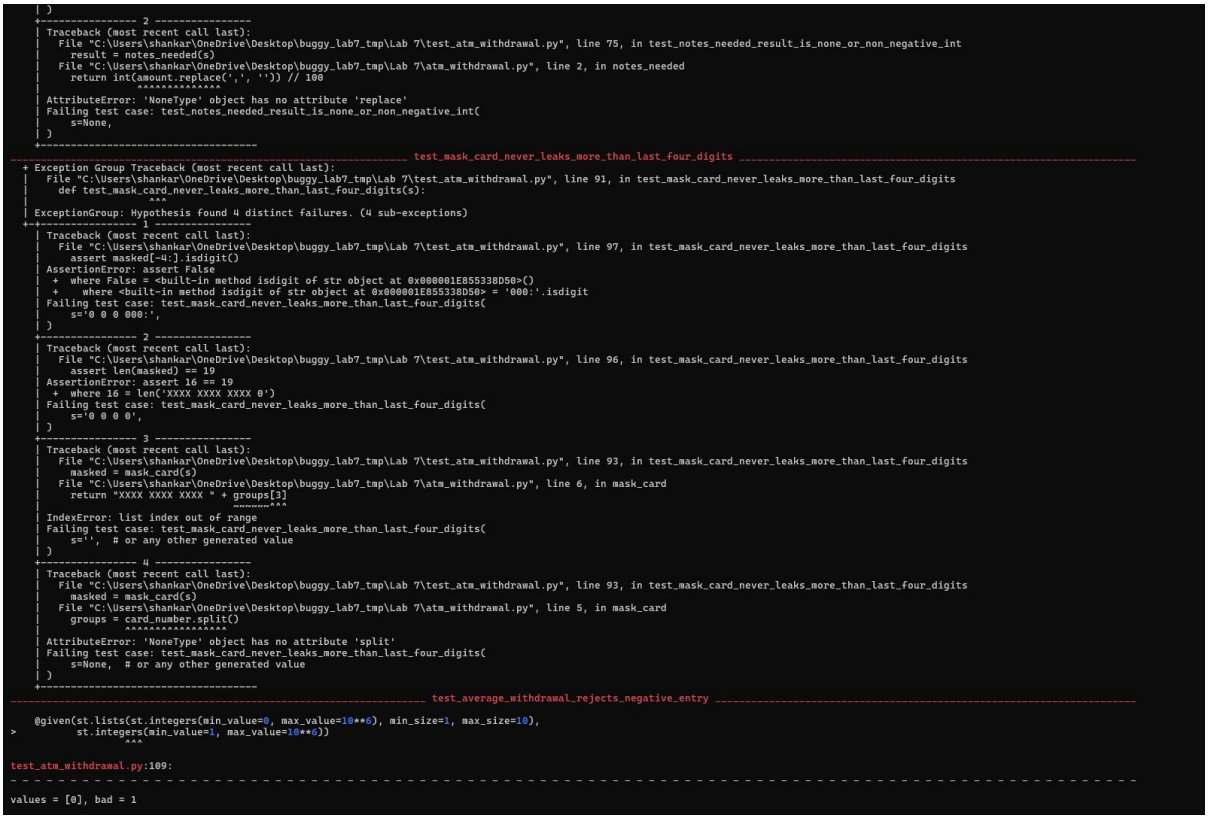


Figure 8: Test cases failing (part 4, mask_card output checks)

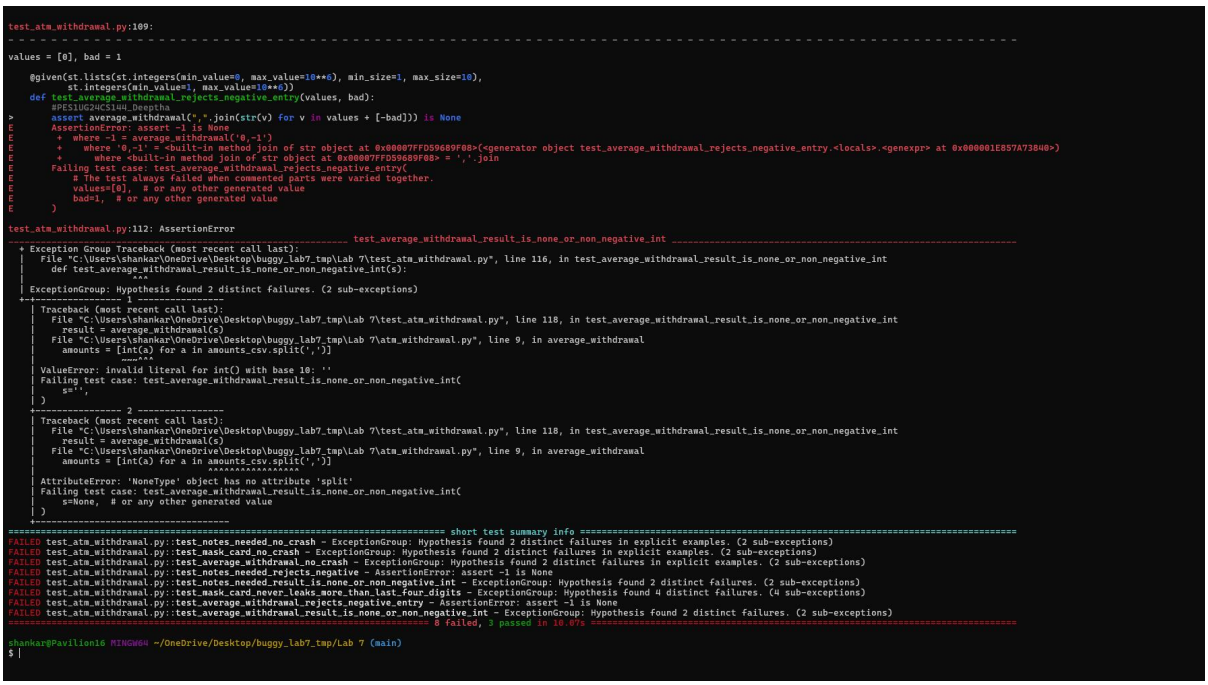


Figure 9: Short test summary: 8 failed, 3 passed

Bugs found

Function	Failing input	Original behaviour	Fix
notes_needed	None	AttributeError	Non-str input returns None
notes_needed	'', ',', 'abc', '2500.50'	ValueError	Only ASCII digits (commas removed) allowed, else None
notes_needed	'-500'	Returns -5 notes (wrong answer, no error)	Negative amounts return None
notes_needed	'1_000'	Silently accepted as 1000	ASCII-digit check rejects it
mask_card	None	AttributeError	Non-str input returns None
mask_card	'', '1234', '1234 5678 9012', '1234567890123456'	IndexError	Require exactly 4 groups
mask_card	'a b c SECRET'	Returns 'XXXX XXXX XXXX SECRET' (leaks arbitrary text)	Each group must be 4 ASCII digits
mask_card	'1234 5678 9012 3456 7890'	Returns '... 3456' (extra group ignored)	Exactly 4 groups required
average_withdrawal	None	AttributeError	Non-str input returns None
average_withdrawal	'', ',', '1,,3', 'one,two'	ValueError	Validate every entry, None if any is bad
average_withdrawal	'-100,200'	Returns 50 (negative withdrawal accepted)	Negative entries return None

Step 5: Fix the application and run it

Every function now validates its input and returns None for anything invalid. main() prints a clear message instead of crashing. Command: python atm_withdrawal.py.

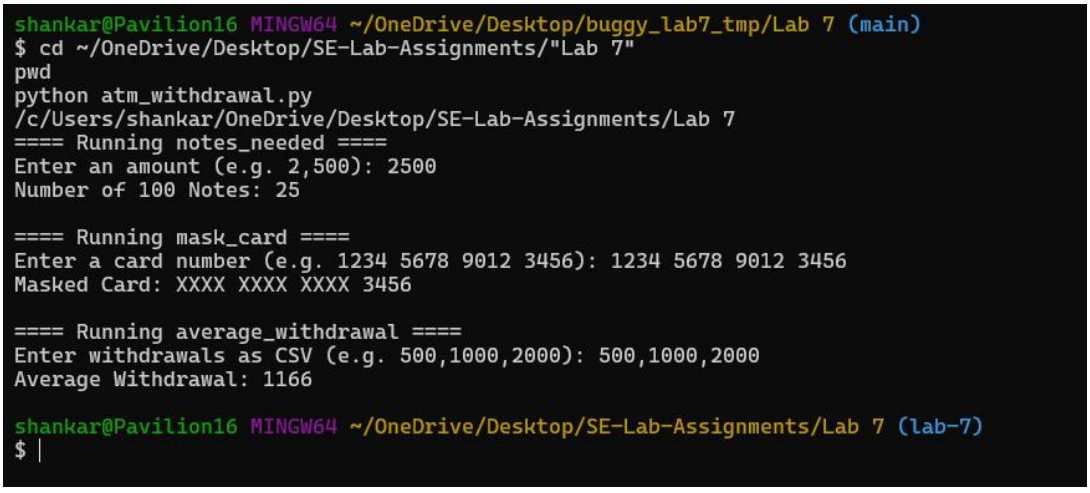


Figure 10: Fixed script with valid input

```
shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/buggy_lab7_tmp/Lab 7 (main)
$ cd ~/OneDrive/Desktop/SE-Lab-Assignments/"Lab 7"
python atm_withdrawal.py
==== Running notes_needed ====
Enter an amount (e.g. 2,500): abc
Number of 100 Notes: Invalid amount

==== Running mask_card ====
Enter a card number (e.g. 1234 5678 9012 3456): 1234
Masked Card: Invalid card number

==== Running average_withdrawal ====
Enter withdrawals as CSV (e.g. 500,1000,2000): 1,,3
Average Withdrawal: Invalid withdrawals list
```

Figure 11: Fixed script handling invalid input without crashing

Step 6: Re-run test_atm_withdrawal.py against the fixed script

Command: python -m pytest test_atm_withdrawal.py. Result: 11 passed.

```
shankar@Pavilion16 MINGW64 ~/OneDrive/Desktop/SE-Lab-Assignments/Lab 7 (lab-7)
$ python -m pytest test_atm_withdrawal.py -p no:langsmith -W ignore
============================= test session starts ==============================
platform win32 -- Python 3.14.0, pytest-9.2.0, pluggy-1.7.0
rootdir: C:\Users\shankar\OneDrive\Desktop\SE-Lab-Assignments\Lab 7
plugins: anyio-4.12.1, hypothesis-6.170.15, langsmith-0.4.34
collected 11 items

test_atm_withdrawal.py ...........                                       [100%]

============================== 11 passed in 1.47s ==============================
```

Figure 12: All test cases passing on the fixed script

Conclusion

Fuzzing with Hypothesis found crashes and silent wrong answers that normal examples missed, including negative amounts being accepted and mask_card returning non-digit text. After adding input validation, all 11 tests pass.